GitHub Copilot and AMP comparison

1. Executive Summary

Both AMP and GitHub Copilot were evaluated across four workflows: **repo scanning**, **task analysis & planning**, **code review**, and **defect triage**. AMP outperformed Copilot in accuracy for tasks requiring cross-repository investigation and deep log analysis, while Copilot offered a more integrated IDE experience with lower friction for standard development tasks. Each tool has distinct strengths suited to different phases of the development lifecycle.

2. Benchmarked Workflows

2.1 Repo Scanner

Metric	GitHub Copilot	AMP (Smart Mode)
Duration	22 min (4:18 pm – 4:40 pm)	16 min total (5:49 – 5:52 base + 5:54 – 6:05 UI libs)
Context window	79.5k / 125k (63.6%)	66% of 168k (~111k)
Model	Claude Opus 4.6	Claude Opus 4.6
Cost	~1 premium request (586.7% 587.7%)	$9.30 (usage-based)
Accuracy	Some drift from actual technical details	More accurate to actual codebase

Notes:

- AMP auto-loaded the `repo-scanner` skill without prompting.
- Post-scan comparison found **AMP's output more faithful to the actual tech stack**; Copilot had minor inaccuracies in framework versions and directory descriptions.

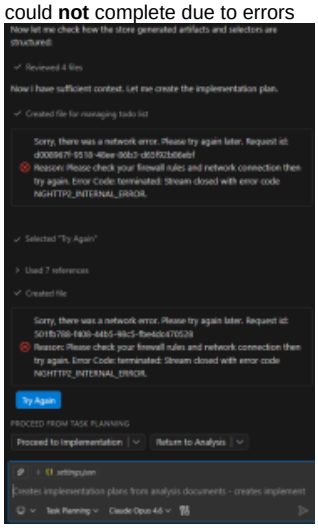
2.2 Task Analysis (ART-86549)

> ***Note:*** *The analysis reports are not directly comparable — different clarification questions were asked even when using the same tool, and the analysis results diverged between runs.*

Metric	GitHub Copilot	AMP
Duration	10 min (5:20 – 5:30 pm)	Comparable
Cost	~1 premium request (588.7% 589.7%)	$11.06
Quality	Full ACs produced; defaulted to "not sure, please check" when prompted about API existence	Full ACs with codebase-grounded answers

2.3 Task Planning

Metric	GitHub Copilot	AMP
Duration	13 min (8:49 – 9:02 am)	Completed

Outcome	could **not** complete due to errors 	Completed the implementation plan successfully using a couple of threads leveraging the built-in handoff tool

2.4 Code Review (ART-86549)

Both tools reviewed the same 22 changed files.

Metric	GitHub Copilot	AMP (Built-in Review with guided checks from code-review agent)*
Total issues	13 (3 high, 6 medium, 4 low)	14 (2 high, 9 medium, 1 low, 2 informational)
Unique finds	`dangerouslySetInnerHTML` XSS risk	Native mobile fingerprinting gap, auto-generated file mutation risk
Overlap	Hardcoded `authBIC8` flagged by both	Hardcoded `authBIC8` flagged by both
Strengths	Clear JSX-based fix suggestions, env-aware analysis	Deeper cross-file dependency analysis, exhaustive-deps violation, missing test case identification

Verdict: Complementary — Copilot caught a security pattern (XSS) that AMP missed; AMP caught architectural risks (native mobile, generated file mutation) that Copilot missed. Running both yields the best coverage.

*Without the guided checks, AMP built-in review did not give actual issues and was not reliable.

2.5 Defect Triage (ART-87579 — eventNotification 403)

Five independent investigation runs were compared.

Metric	GitHub Copilot + Splunk MCP	AMP (Librarian + Splunk MCP)
Duration	~10 min	~10 min
Root cause accuracy	**Partial** — correct mechanism (JWT audience mismatch) but wrong primary fix location (this repo vs. external widget repo)	**Correct** — identified external `pingmfa-v1-widget` repo as primary fix target
Cross-repo investigation	Not performed (no Bitbucket access)	Extensive — read external repo configs via Librarian
Splunk evidence	146 events, 9 correlation IDs	4 distinct log sources (JWT attrs, Kong, Nginx DMZ, Akamai CDN)
PROD impact	Missed	Identified active production 403s with definitive CDN evidence
Secondary findings	None	South-edge entitlements gap in `pingmfa-v1`
Fix options	1 (local config change — would not fully resolve)	3 options (gateway realignment, token exchange, accepted audience registration)
Confidence calibration	10/10 on incomplete fix (false positive)	Appropriately calibrated

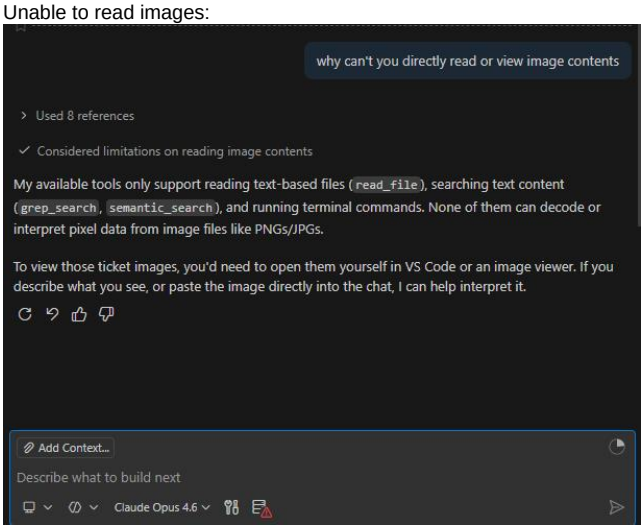
Verdict: AMP's Librarian tool (cross-repo code access) was the decisive differentiator. Copilot's fix would have left the defect unresolved because it could not investigate the external widget repository where the root cause lived.

3. Strengths & Weaknesses

3.1 GitHub Copilot — Strengths

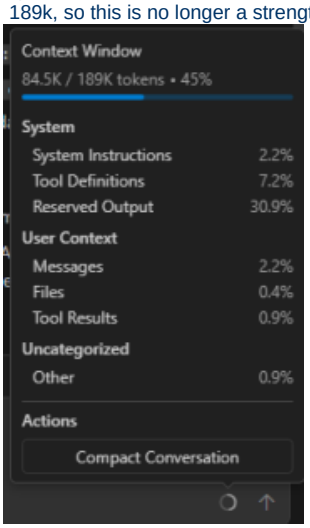
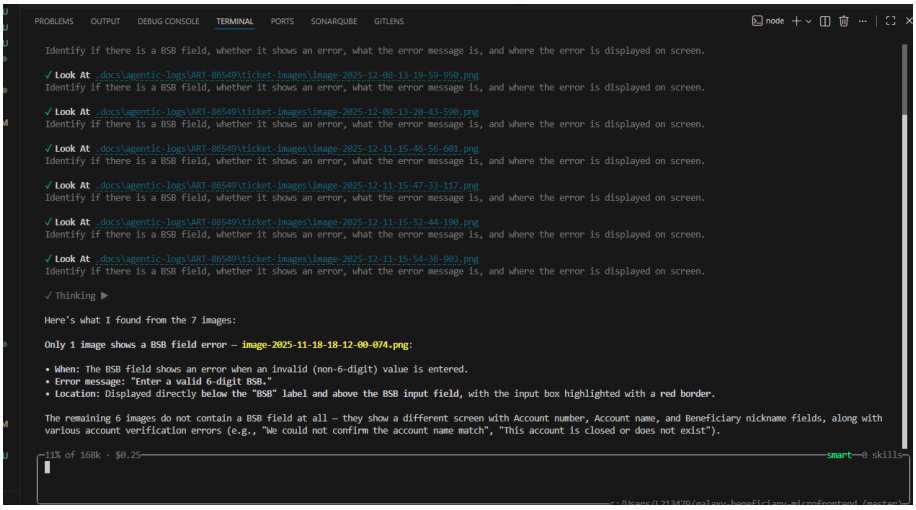
Strength	Detail
IDE integration	Native VS Code experience — inline completions, chat panel, diff highlighting with accept/reject per hunk
Diff visibility	Clear highlighting of changed sections; easy to keep/undo individual changes before committing
Model flexibility	Can switch between models (Claude, GPT-4o, o3, Gemini) per conversation
Instructions system	Structured `.github/instructions/` with `applyTo` glob patterns for context-aware guidance
Pre-set prompts	`.github/prompts/` files for repeatable workflows (code review, defect triage, fix tests)
Agent ecosystem	`.github/agents/` with reusable mode definitions and tool access
Cost model	Cheaper than AMP — flat subscription with premium request allocation vs. AMP's per-use pricing (e.g., $9–11 per complex task)

3.2 GitHub Copilot — Weaknesses

Weakness	Detail
No remote repo access	Cannot read external repositories; critical limitation for microfrontend/widget architectures
Image analysis	Organisation restriction / proxy setting resulting into inability to read images. Unable to read images: 

3.3 AMP — Strengths

Strength	Detail
Cross-repo investigation	Librarian tool provides native Bitbucket codebase access — decisive for accurate root cause analysis

Context window	168k (smart mode); even larger in deep mode (GPT-5.3 Codex) — avoids context exhaustion on complex tasks Note: Copilot's context window may increase in future, narrowing this gap Update on 11 Mar 2026 Latest vscode version 1.110.1, Copilot Claude Opus 4.6 context window increased to 189k, so this is no longer a strength of AMP as mentioned in the note above. 
Handoff tool	Passes context between threads for multi-session workflows
Built-in code review	`code-review skill` with configurable checks (security, code-standards, testing)
Image/PDF reading	Native image analysis in smart mode via built-in vision tool 
Walkthrough skill	Generates architectural diagrams (Mermaid) with strong accuracy
Oracle	"Second opinion" capability for task planning validation (requires workflow adaptation to use)

3.4 AMP — Weaknesses

Weakness	Detail
Handoff hallucination risk	Context passed between threads may contain inaccuracies; also consumes ~10% of the receiving thread's context window
No model selection	Locked to modes (smart/deep/rush) — cannot choose a specific LLM model
No diff highlighting	Does not visually highlight changed sections or offer per-hunk accept/reject for uncommitted new files
`.env` handling	Claims inability to read `.env` files directly, but works around it by scripting `Get-Content`
Tool availability by mode	Some built-in tools (e.g., image reading) only available in **smart** mode; deep mode resorts to OCR shell commands

No pre-set prompts	Prompt concept merged into skills; no standalone reusable prompt files
No structured instructions	No `applyTo`-based instruction system; guidance scattered across `AGENTS.md` files and skill definitions (mitigated by `@`-references from `AGENTS.md`)
Trigger discoverability	Built-in tools activated by specific phrases (e.g., "look at" triggers image reading) — **requires learning the conventions, though this becomes natural with use**

3.5 Open Questions

Topic	Detail
Maintenance effort	AMP's design places `AGENTS.md` in each subdirectory (only loaded when that folder is accessed). This keeps files small and focused but scatters governance across the tree. AMP can assist with keeping these files updated, partially offsetting the maintenance burden. Whether this is a blocker depends on team size and convention discipline.

4. Feature Comparison Matrix

Capability	GitHub Copilot	AMP
IDE inline completions	Yes	No
Chat-based code generation	Yes	Yes
Diff highlighting & accept/reject	Yes	No
Model selection	Yes (multiple)	No (mode-based)
Context window	Varies by model. Update from 11 Mar 2026 ~~125k~~ 189k (Claude Opus 4.6 to compare with AMP smart mode) 272k (GPT 5.3-codex to compare with AMP deep mode)	168k (smart mode) 272k (deep mode)
Cross-repo code access	No	Yes (Librarian)
Image/PDF analysis	Limited	Yes (smart mode)
MCP server integration	Yes	Yes
Pre-set prompt files	Yes	No (merged into skills)
Structured instructions (`applyTo`)	Yes	No (`AGENTS.md` refs)
Built-in code review	No (via agent/prompt)	Yes (`amp review`)
Multi-session handoff	No	Yes (with caveats)
Architectural diagramming	Basic	Strong (walkthrough)
Spec-driven workflow	Manual	Built-in (5-phase)
Oracle / second opinion	No	Yes

5. Cost Model Comparison

Factor	GitHub Copilot	AMP
Pricing model	Subscription + premium requests	Usage-based (per context consumed)
Repo scanner cost	~1 premium request	US$9.30
Task analysis cost	~1 premium request	US$11.06
Cost visibility	Premium request counter (% of allocation)	Dollar cost + context % per session

6. Recommendations

Category	Task	Tool	Rationale

One-off / periodic	Repo scanning	AMP	Higher accuracy, auto-skill loading — **higher cost** justified by infrequent use
	Architectural documentation	AMP	Built-in **walkthrough** skill generates Mermaid diagrams with strong accuracy
Simple tickets	Analysis, planning & implementation, unit test fixes	Copilot	**Cheaper** per interaction; IDE inline completions, **diff highlighting**, **autocomplete**, **model flexibility**
Complex tickets	Analysis, planning	AMP	Built-in **handoff tool** passes context between threads for continuous multi-session delivery
Visual assets	Requirements with mockups / PDFs	AMP	Built-in **image reading** tool in smart mode; Copilot has limited image/PDF capability
Code review	PR review	Both	Complementary — Copilot catches security patterns (XSS), AMP catches architectural risks. Run both
Defect triage	Investigation & log analysis	AMP	Librarian cross-repo access + deep Splunk integration across multiple sourcetypes